

.NET Framework

The first version of Microsoft's .NET platform (2002). Works only on Windows. Used for ASP.NET Web Forms, WPF, old ASP.NET MVC apps. Analogy: Like a train that runs only on one track (Windows).

Example:

ASP.NET MVC application → web.config file → IIS server → Windows only.

Keywords: CLR, Windows-only, web.config, legacy

.NET Core

Modern, cross-platform, open-source version of .NET (2016). Runs on Windows, Linux, Mac. Used for modern web apps, APIs, cloud apps. Analogy: Like a car that runs on all roads (any OS).

Example:

```
dotnet new webapi -n MyApi
```

Keywords: cross-platform, lightweight, open-source, cloud

Configuration: web.config vs appsettings.json

web.config (Framework): XML format, single large file. appsettings.json (Core): JSON format, cleaner and supports environment configs.

Example:

```
<appSettings>
  <add key="AppName" value="MyApp"/>
</appSettings>

{
  "AppName": "MyApp",
  "Logging": { "LogLevel": "Information" } }
```

Keywords: XML, JSON, configuration, environment

Value Types vs Reference Types

Value Types: Store value directly (stack). Reference Types: Store memory address (heap).

Analogy: Value = photocopy, Reference = shared notebook.

Example:

```
int x=10; int y=x; y=20; // x still 10

int[] arr1={1,2}; int[] arr2=arr1; arr2[0]=99; // arr1[0]=99
```

Keywords: stack, heap, copy, reference

Project Reference

Allows one project to use code from another project. Example: Web project reuses MathLibrary.dll.

Analogy: Borrowing notes instead of rewriting.

Example:

In Visual Studio: Right-click project → Add Reference

Keywords: DLL, dependency, reuse

Types of Classes

Concrete: Normal class. Abstract: Must be inherited. Static: Only static members. Sealed: Cannot be inherited. Partial: Split across files.

Example:

```
abstract class Animal { public abstract void Speak(); }
class Dog : Animal { public override void Speak()=>Console.WriteLine("Bark"); }
```

Keywords: abstract, static, sealed, partial, inheritance

Types of Methods

Instance: Belongs to object. Static: Belongs to class. Abstract: Declared but no body. Virtual: Can be overridden. Extension: Extend existing classes.

Example:

```
class Calculator {
    public int Add(int a,int b)=>a+b;
    public static int Multiply(int a,int b)=>a*b;
}
```

Keywords: instance, static, abstract, virtual, extension

Constructors and Types

Special methods run when object created. Types: Default, Parameterized, Copy, Static.

Example:

```
class Student { public string Name; public Student(){Name="Unknown";} public Student(string n){Name=n}; }
```

Keywords: default, parameterized, copy, static

Access Modifiers

Control visibility. public: anywhere. private: only inside class. protected: class + derived. internal: same project.

Example:

```
class Person { private int age; public string Name; }
```

Keywords: public, private, protected, internal

Method Overloading vs Method Overriding

Overloading = same name, different params (compile-time polymorphism). Overriding = derived class changes base method (runtime polymorphism).

Example:

```
class MathOps{ public int Add(int a,int b)=>a+b; public double Add(double a,double b)=>a+b; }

class Animal{ public virtual void Speak()=>Console.WriteLine("Sound"); }
class Dog:Animal{ public override void Speak()=>Console.WriteLine("Bark"); }
```

Keywords: compile-time, runtime, polymorphism, override